PRACTICAL LABORATORY REPORT

Local AI with Ollama

Running, evaluating and integrating local language models

Author	Loubna El Ghazi
Project	FSTT Local AI Assistant
Models	`qwen3.5:4b` and `gemma3:4b`
Stack	Python, Ollama, HTTP/JSON, Flask and local document retrieval
Date	21 October 2026

Abstract

This laboratory investigates how language models can run locally and be integrated into a Python application. Experiments cover prompt design, model management, generated-code validation, conversation history, JSON Schema, retrieval-augmented generation and image analysis. The final assistant combines a Flask interface, course-document retrieval, conversational context and optional structured responses.

In the latest recorded cold-start benchmark, Gemma completed the request in 20.03 s compared with Qwen's 47.25 s and used 2.88 GB rather than 3.14 GB of GPU memory. Both generated primality functions matched the reference for all 2,005 integers from -5 to 1,999. These results support task-specific model selection while highlighting the need to verify generated content and distinguish loading costs from generation speed.

01 / SETUP AND FOUNDATIONS

Environment and model management

Objective and execution environment

The objective was to run inference on the machine hosting Ollama, call the service from Python, evaluate two local models, and develop a course assistant. The recorded service check returned `Ollama is running`. The installation output identifies Ollama version `0.40.7`. Requests targeted `http://localhost:11434`.

The application requires Python 3.10 or newer. The exact Python version, operating-system version, CPU, GPU model and total installed RAM were not recorded in the experimental results. The inference counters provide evidence of GPU-memory allocation for both tested models. Lab notebook, p. 2, 15, 53.

Downloaded model inventory

Model	Parameters	Disk (GB)	Quantization
`qwen3.5:4b`	4.2B	3.32	`Q4_K_M`
`gemma3:4b`	3.9B	3.35	`Q4_K_M`
`qwen2.5-coder:7b`	7.6B	4.68	`Q4_K_M`

The coder model was present in the inventory but was not included in the two-model comparison. Storage quantities use decimal GB, matching the notebook's division by 10^9.

Downloaded versus loaded

`ollama list` reports model files stored on disk, with names, identifiers, sizes and modification dates. `ollama ps` reports models currently resident for inference, including allocation and remaining keep-alive time. A downloaded model persists after a restart; a loaded model occupies RAM and/or VRAM and can be unloaded while its files remain on disk. The initial API snapshot showed only Qwen loaded, with 3.14 GB reported in VRAM.

Local inference and quantization

Local AI runs computation on the user's own machine instead of a provider's remote inference server. Python still uses an API, but its destination is a local HTTP service. Once model files are downloaded, this configuration can operate without Internet connectivity. A separate disconnected-network run was not recorded; offline operation was therefore not independently verified in this experiment.

Quantization reduces the precision used to represent weights. `Q4_K_M` is a scheme using mainly four-bit weights, reducing disk and memory requirements with a possible quality trade-off. Model weights are only one part of inference memory: context caches, activations and runtime buffers also contribute.

Local processing improves control over prompts and documents. It does not remove hardware costs, model errors, application vulnerabilities or the need to check generated answers.

02 / PROMPTING AND CODE

Prompt design and programming evaluation

Audience, role and response format

Qwen received three requests explaining AI: a five-sentence definition, an explanation for a ten-year-old, and an explanation for a computer-science professor. It changed vocabulary, examples and technical depth. The first two recorded response times were 31.9 s and 28.1 s. Its child-oriented explanation also made an inaccurate claim suggesting ordinary programs cannot add new numbers without a newly supplied rule; adapting style does not guarantee factual accuracy.

The DAG exercise produced a formal explanation with topological ordering in 63.9 s and a more conversational explanation using a sandwich dependency example in 41.5 s. These differences illustrate the following prompt controls:

Control	Effect on the response
Role	Establishes perspective and tone, such as a professor or code reviewer.
Context	Identifies audience, task and relevant background.
Constraints	Set length, scope and required inclusions or exclusions.
Output format	Specifies a structure such as definition, example, code or JSON.
Specificity	Reduces ambiguity and makes compliance easier to evaluate.

The timings describe these individual outputs; they do not isolate prompt complexity from answer length, loading or other runtime effects.

Two-model primality task

Both models were asked for a Python primality function with type hints, a docstring, an algorithm explanation and time complexity.

Measurement	Qwen	Gemma
Generation request time (s)	16.6	12.8
Notebook response-quality score /5	4	4
Notebook code-correctness score /5	5	4
Reference agreement (−5 through 1,999)	2,005/2,005	2,005/2,005

Qwen eliminates multiples of two and three, then checks $6k \pm 1$ candidates with an integer condition `i*i <= n`. Gemma tests every integer divisor through `int(n**0.5)`. Both have $O(\sqrt{n})$ trial-division time and $O(1)$ auxiliary variables under the conventional fixed-cost arithmetic model. Qwen's version performs fewer division checks and avoids floating-point square-root limits for very large integers.

Both candidates successfully matched the reference implementation. These finite tests establish behavior only over that interval. The subjective score difference reflects implementation robustness, not failed test cases. Qwen's prose also contains a mathematical error: the correct fact is that every composite integer n has a nontrivial factor at most $\sqrt{n}$. Lab notebook, p. 17–22.

03 / APPLICATION BEHAVIOR

Conversation, code review and structured data

Conversation history is maintained by the application

In the memory experiment, the model correctly answered that the user's name was Ahmed. The printed prompt showed why: the application sent the earlier statement, the assistant's reply and the new question together. Another conversation similarly repeated the supplied name and research field.

```
User: My name is Ahmed.
Assistant: Nice to meet you, Ahmed! How can I assist you today?
User: What is my name?
Assistant:
```

This behavior is contextual recall, not a persistent personal memory written into model weights. With `/api/chat`, the same principle uses a list of `system`, `user` and `assistant` messages. The application must retain and resend relevant turns; clearing that history removes their availability in a new request. Lab notebook, p. 33–34.

Software-engineering assistance

The lab also submitted Java examples for review. The calculator exercise concerns division by zero and the truncation behavior of integer division. In the student-class example, `age = age;` assigns a parameter to itself; updating the object's field requires `this.age = age;`. These examples demonstrate useful review assistance, but generated patches still require compilation, tests and inspection of their intended behavior.

Prompt-only JSON versus JSON Schema

The student-analysis request supplied a name, age, grade and absence count. A prompt asking for JSON produced parseable output in 1.7 s, but used `"risk": "Low"`. The later schema restricted risk to lowercase values `low`, `medium` and `high`. Schema-based generation produced `"risk": "low"`, and the grade 15.5 parsed as a Python `float`. Lab notebook, p. 40–41.

Property	Prompt-only output	Schema-based output
Required fields	Present in this example	Declared in schema
Risk value	`Low`	`low`
Grade	15.5	15.5; parsed as `float`
Integration	Needs parsing and checks	Predictable fields and types

Structured output enables automated validation and downstream use. It constrains representation, not truth. No explicit grading policy or risk threshold was supplied, so the academic-risk judgment is not validated merely because it conforms to a schema. The regex extraction helper also does not constitute general schema validation: multiple objects or malformed output may require more careful handling.

04 / DOCUMENTS AND VISION

Grounding answers in supplied information

Whole-document questioning

The initial course file described machine learning, labelled and unlabelled learning, and reinforcement learning. For the supported question, the model returned the course definition of reinforcement learning in 13.0 s. For quantum computing, which was absent from the document, it returned `Information not available.` in 0.2 s. The instruction was followed in both examples; two examples do not establish universal reliability.

Retrieval-augmented generation

The next exercise indexed local text files with token counts, stop-word filtering and inverse-document-frequency weighting. The retriever ranked documents by overlapping question terms, selected the top result, and inserted it into the prompt. The actual index included `course.txt` alongside the four topic documents, explaining why the general course file was selected for two questions.

Question / observed answer	Retrieved file	Time (s)
Reinforcement learning / rewards and penalties	`course.txt`	15.0
Training hardware / GPUs	`deep_learning.txt`	0.3
Supervised learning / labelled data	`course.txt`	0.3
2018 World Cup / information unavailable	None	0.3

The World Cup question is deliberately outside the course. Abstention is the desired behavior even if the model could answer from its pretrained knowledge. These are individual request timings, not a controlled latency comparison. Lab notebook, p. 45–46.

Keyword retrieval can miss synonyms and context-dependent follow-ups. An optional semantic-retrieval extension was implemented using embeddings. Its retrieval accuracy was not evaluated in the recorded results.

Optional multimodal experiment

Gemma received an image as base64 data with a text request identifying objects, environment, visible text and possible interpretation. It generated an analysis in 47.4 s, describing a laptop and its surrounding scene.

A text-only LLM receives text; a vision-capable multimodal model also processes image information. Specific device variants, screen text and inferred intent in its answer need visual verification and are not treated as established facts. Local applications include OCR, document analysis, industrial inspection and educational assistance.



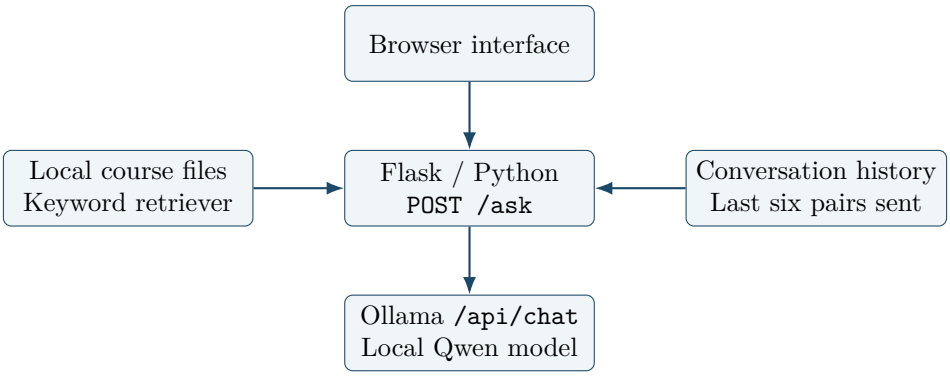
Input image used in the vision experiment.

05 / FINAL MINI PROJECT

FSTT Local AI Assistant

Architecture and request flow

The final application combines a local Flask interface with the same Ollama service. Its browser interface is served at `127.0.0.1:5001`; the model endpoint remains `localhost:11434`. The notebook launch explicitly sets `MODEL=qwen3.5:4b`.



For each question, the backend retrieves two documents, creates a context-bearing message and appends it to the system instructions and recent history. It sends this list to Ollama, optionally supplies a JSON Schema, and returns the answer, sources, seconds, token count and throughput. A reset endpoint clears the in-process conversation history.

Recorded integration tests

Test and observed behavior	Time (s)	Tokens/s
Course question: reinforcement learning definition	2.07	57.8
Follow-up: comparison with supervised learning	0.77	61.7
Structured deep-learning summary: JSON object	2.47	50.4
Unsupported quantum-computing question: abstention	0.90	67.1

The structured response contained `topic`, `summary`, `key_points` and `difficulty`. The unsupported question returned an empty source list and `Information not available.`. The server was subsequently stopped successfully. Lab notebook, p. 60–61.

Requirement coverage and limits

The application implements local execution, course retrieval, conversation history, structured output, answer display and timing. The integration tests demonstrate these functions through recorded requests and responses. Appendix A presents representative execution results.

The history list is global and intended for a single-user demonstration. Only the last six pairs are sent, while the stored list itself is not bounded until reset. Concurrent users would need separate sessions. Retrieval uses the current question without history-aware rewriting, so the successful follow-up test does not establish reliability for every ambiguous follow-up.

06 / EXPERIMENTAL COMPARISON

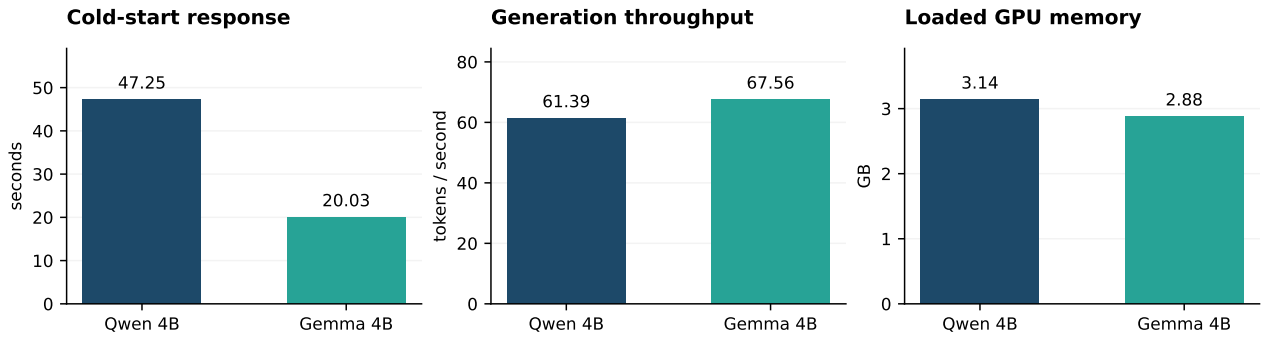
Cold-start benchmark results

Protocol

Part 16 used the same prompt for both models: *Write a 500-word explanation of reinforcement learning.* Each model was requested once after an unload request and a two-second wait. Generation was non-streaming with `think=False`. Wall-clock duration measured the generation HTTP request; load duration and generation counters came from Ollama. Memory was sampled after the response.

Metric	Qwen3.5:4b	Gemma3:4b
Disk size (GB)	3.32	3.35
Loaded size (GB)	3.14	2.88
VRAM (GB)	3.14	2.88
Load time (s)	35.11	7.97
Response time (s)	47.25	20.03
Generated tokens	731	800
Generation throughput (tokens/s)	61.39	67.56
System RAM used (%)	48.2	58.8
Notebook-assigned quality /5	4	4

Lab notebook, p. 53.



Interpretation

Gemma completed the measured request approximately 47.25/20.03 = 2.36 times as fast overall, generated tokens about 10.1% faster and used 0.26 GB less VRAM (about 8.3% less than Qwen). Qwen's loading phase accounts for about 74% of its response time. Subtracting the reported load duration leaves 12.14 s for Qwen and 12.06 s for Gemma; these residuals still include prompt processing and request overhead and are not pure decoding durations.

Both models were assigned a qualitative score of 4/5 in the notebook. These manual ratings are distinct from the automatically measured timing and memory metrics. Token counts alone do not establish compliance with the requested 500 words.

The values above describe a single cold-start run per model. Repeated runs are needed to assess variability and distinguish model-loading effects from steady-state inference performance.

07 / DISCUSSION

Selection, limitations and deployment

Choosing between the tested models

Gemma is the stronger initial choice for this machine when cold-start response time and GPU-memory footprint are priorities. Its disk file is slightly larger, illustrating that disk size alone does not predict runtime memory or latency. Qwen remains attractive for the programming example because its generated algorithm performs fewer divisibility checks and uses integer arithmetic, although its explanation needed correction.

For limited system RAM, the 2.88 GB loaded footprint suggests trying Gemma first, but these tests used GPU memory. A CPU-only or smaller-VRAM target must be measured separately. If neither model fits comfortably, a smaller model or another quantization should be evaluated against the actual task. The largest model is not necessarily the best practical choice.

Experimental limits

- One run per model provides no estimate of variability. Model order, previous resident models, disk caches and background activity can affect results.
- The unload helper requests eviction but does not explicitly verify it through `/api/ps`. Hardware identifiers, driver versions and full runtime specifications were not recorded.
- System RAM percentages are host-wide snapshots, not isolated per-model peak usage. Tokenizers and output lengths can differ between models.
- Prompt length, context-window settings and sampling configuration are not fully standardized. Quality scores are subjective and benchmark answer text was not retained.
- Finite code tests and a few grounding examples do not prove universal correctness. The disconnected-network test and a completed semantic-retrieval result are not separately documented.

A stronger follow-up experiment would record hardware and exact model versions, retain answers, control settings, separate warm and cold runs, and repeat each condition. Median latency, peak process/VRAM usage and a predefined quality rubric would make comparisons more reproducible.

Network security and application isolation

The final Flask process binds to loopback with debugging disabled. Exposing Ollama on `0.0.0.0` would make it listen on all interfaces; actual external reachability then depends on routing and firewall rules. Without authentication, reachable users could consume compute or access exposed model-management operations. Unencrypted requests, logs and application storage can disclose sensitive information.

For authorized remote use, restrict network access and place the service behind authentication and TLS. These controls protect access and traffic, not model reasoning: malicious instructions in retrieved documents can still cause prompt injection. Treat document text as untrusted data, limit tool permissions, validate outputs, and isolate conversation state per user. The demo's shared history is unsuitable for a multi-user service without changes.

08 / REQUIRED REPORT ANSWERS

Questions 1–8

Q1. What is Local AI?

Local AI runs models on the user's own computer or locally controlled server. In this lab, Python sends prompts to a local Ollama process, which performs inference on the host's CPU and/or GPU.

Q2. Difference between a Cloud LLM and a Local LLM?

A cloud LLM runs on a provider's remote infrastructure and is accessed over a network. A local LLM runs on the user's hardware. Local execution offers control over prompts and offline use after setup, while cloud services can provide remotely managed compute and models beyond the local machine's capacity.

Q3. What role does Ollama play?

Ollama manages model downloads, loading and execution. Its local HTTP API provides a common interface for the terminal, Python scripts and the Flask application to access the same model.

Q4. What is model quantization?

Quantization stores weights with fewer bits to reduce model storage and memory requirements. Both compared models used `Q4_K_M`. Reduced precision can affect quality, so quantized models must still be tested on the intended tasks.

Q5. Why does model size affect memory consumption?

More parameters normally require more space for weights at a given precision. Inference also allocates context caches and working buffers, so memory depends on parameter count, quantization, context length and execution configuration rather than disk size alone.

Q6. Why can a smaller model sometimes be preferable?

A smaller model can fit available memory, load faster and provide acceptable task quality with fewer resources. It can therefore deliver a better user experience on constrained hardware even when a larger model is more capable on some tasks.

Q7. Advantages of local inference?

Benefits include control over sensitive data, operation without Internet once dependencies and model files are available, customizable application behavior and no per-token cloud API charges. The user still bears compute, electricity and maintenance costs.

Q8. Disadvantages of local inference?

Local models consume storage and memory and may respond slowly on limited hardware. The user must manage installation and updates. Model capacity is hardware-limited, generated content can be wrong, and local hosting still requires application and network security.

Offline functionality is supported by the local architecture; a separate disconnected execution was not recorded in these experiments.

09 / REQUIRED REPORT ANSWERS

Questions 9–15

Q9. Explain the architecture of your Python application.

The browser calls Flask. Python retrieves two relevant course files, combines context with system instructions and recent conversation messages, and calls Ollama's local chat API. Flask returns the answer, source names and performance metrics. Selected requests use a JSON Schema. Section 05 presents the architecture and observed integration tests.

Q10. Explain the role of the REST API.

HTTP endpoints connect the application to Ollama using JSON. `/api/generate` accepts generation prompts, `/api/chat` accepts role-based messages, `/api/tags` lists downloads and `/api/ps` reports loaded models. The API is local in this setup; using HTTP does not imply cloud inference.

Q11. What is RAG?

Retrieval-augmented generation finds relevant external information and supplies it as context for generation. Here, keyword-based ranking selects local course text. This changes the request context without retraining the language model. The unsupported test correctly produced an unavailable-information response.

Q12. Difference between an LLM and a multimodal LLM?

A text-only LLM processes text. A multimodal model additionally accepts inputs such as images, so it can reason about visible objects, layout and text. Visual interpretations remain predictions and may include unsupported details.

Q13. Compare two local models experimentally.

The complete latest Part 16 table is reproduced in Section 06. Gemma's response took 20.03 s versus 47.25 s for Qwen, with throughput of 67.56 versus 61.39 tokens/s and VRAM usage of 2.88 versus 3.14 GB. Both received a manually assigned quality score of 4/5. In Part 6, Gemma generated code in 12.8 s versus 16.6 s for Qwen; both primality functions passed all 2,005 reference comparisons. Qwen's implementation was more optimized, while its explanation contained an error.

Q14. Which model would you deploy on a computer with limited RAM?

Between these candidates, I would initially evaluate Gemma because its measured loaded footprint was lower and its requests were faster. These measurements are GPU-based, so they do not establish CPU-only RAM use. I would verify fit and quality on the target hardware, selecting a smaller quantized model if necessary.

Q15. What security risks exist when exposing a local LLM server?

An unauthenticated reachable service can permit unauthorized inference and resource consumption, or expose model-management operations. Prompts, responses and shared histories can leak through insecure transport, logs or application bugs. Retrieved documents can carry prompt injection. Retain loopback binding where possible; otherwise apply firewall restrictions, authentication and TLS, isolate users, validate outputs and restrict tools. Network protection alone does not prevent prompt injection.

APPENDIX A / EXECUTION EVIDENCE

Installation, models and conversation

The following excerpts document installation, model management, conversation history, code validation, retrieval and application behavior.

A1. Local service reachable

```
!ollama --version

]11;?\a version is 0.40.7
```

Make sure the Ollama service is running (the desktop app starts it automatically; otherwise run `ollama serve` in a separate terminal).

```
import requests

OLLAMA = "http://localhost:11434"

try:
    r = requests.get(OLLAMA, timeout=3)
    print("✅ Ollama server reachable:", r.text)
except requests.exceptions.ConnectionError:
    print("❌ Ollama is not running. Start it with `ollama serve` in a
terminal.")

✅ Ollama server reachable: Ollama is running
```

Source: lab notebook, p. 2. Version and local service connectivity check.

A2. Downloaded models and GPU allocation

```
Downloaded:

                name  size_GB  family params quantization
0        qwen3.5:4b     3.32  qwen35   4.2B       Q4_K_M
1         gemma3:4b     3.35  gemma3   3.9B       Q4_K_M
2  qwen2.5-coder:7b     4.68   qwen2   7.6B       Q4_K_M

Currently loaded in memory:

         name  size_GB  in_VRAM_GB                          expires_at
0  qwen3.5:4b     3.14        3.14  2026-10-21T14:28:53.49077485+01:00
```

Source: lab notebook, p. 15. Model inventory and the Qwen loaded-memory snapshot.

A3. Conversation history supplied in the prompt

```
You: My name is Ahmed.
AI : Nice to meet you, Ahmed! How can I assist you today?

You: What is my name?
AI : Your name is Ahmed!


print("What the model actually received on the last call:\n" + "-"*50)
print("\n".join(history[:-1]) + "\nAssistant:")

What the model actually received on the last call:
--------------------------------------------------
User: My name is Ahmed.
Assistant: Nice to meet you, Ahmed! How can I assist you today?
User: What is my name?
Assistant:
```

Source: lab notebook, p. 33. Name recall and the exact prior turns included in the next request.

APPENDIX A / EXECUTION EVIDENCE

Code validation, RAG and final application

A4. Generated Python functions checked against a reference

```
{mismatches}"
    print(f"{name} matches the reference from -5 to 1999")

qwen3.5:4b matches the reference from -5 to 1999
gemma3:4b matches the reference from -5 to 1999
```

Source: lab notebook, p. 22. Both model functions matched the reference from −5 to 1,999.

A5. Retrieved documents and grounded answers

```
Q: What is reinforcement learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Reinforcement learning learns through interaction with an
environment using rewards and penalties.
  (15.0s)

Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 8.42789552798877)]
  A: GPUs.
  (0.3s)

Q: What is supervised learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Supervised learning uses labelled data.
  (0.3s)
```

Source: lab notebook, p. 45. Retrieved course documents, supported answers and individual timings.

A6. Final application's structured and unsupported requests

```
Q: Summarize deep learning
A: {'difficulty': 'easy', 'key_points': ['Uses artificial neural
networks with many layers', 'Handles large amounts of data', 'CNNs for
images', 'Recurrent networks/transformers for sequences and language',
'Requires GPUs for training'], 'summary': 'Deep learning uses multi-
layered artificial neural networks to learn representations from large
datasets, specifically employing convolutional neural networks for
images, recurrent networks and transformers for sequences and
language. Training typically requires GPUs.', 'topic':
'deep_learning.txt'}
   sources=[{'doc': 'deep_learning.txt', 'score': 2.43}, {'doc':
'course.txt', 'score': 0.21}] · 2.47s · 50.4 tok/s

Q: What is quantum computing?
A: Information not available.
   sources=[] · 0.9s · 67.1 tok/s
```

Source: lab notebook, p. 61. Recorded responses from the Flask application: structured summary with source metadata, followed by abstention with no sources.

CONCLUSION

Findings and final assessment

This laboratory demonstrated how local language models can be accessed through Ollama and integrated into a complete Python application. The work progressed from basic generation and prompt design to conversation history, structured JSON output, local document retrieval and a Flask course assistant. The final application answered supported questions, handled a conversational follow-up, returned a structured summary and declined an unsupported question in the recorded tests.

The experiments showed that model selection should depend on measurements and task requirements. In the latest cold-start benchmark, Gemma3:4b responded in 20.03 s compared with 47.25 s for Qwen3.5:4b and used less VRAM: 2.88 GB instead of 3.14 GB. Much of the latency difference came from model loading, while generation throughput was closer. For programming, both generated primality functions passed all 2,005 reference comparisons, but Qwen produced the more optimized implementation. Its incorrect mathematical explanation nevertheless showed why correct-looking output still requires review.

The application experiments also clarified the responsibilities of the surrounding software. Conversation memory came from messages retained and resent by Python, not from changes to model weights. JSON Schema made response structure more predictable, while RAG supplied course knowledge without retraining. Neither mechanism guarantees factual correctness, and the successful examples should not be interpreted as proof of universal reliability.

Overall, the project provides a working demonstration of a locally hosted educational assistant with control over its document context and inference environment. Gemma is a reasonable initial choice when measured latency and GPU-memory usage are priorities, while the final Qwen-based assistant also demonstrated the required functionality. Further evaluation should repeat benchmarks, retain generated answers for quality assessment, document the target hardware and test a wider set of questions. Before multi-user deployment, conversation state should be isolated and access controls strengthened.

The main outcome is a practical understanding that a useful local AI system depends on more than the model: retrieval, prompt design, validation, application state and hardware constraints all shape its behavior.